Backend Engineer — Take-Home Assessment

Build a legal precedent research agent that can analyze a corpus of court judgments and provide strategic case analysis.

About Lexi

Lexi builds AI-powered associates for law firms. Our platform handles legal research, document drafting, contract review, and case analysis using a multi-agent orchestration architecture. We are backed by Y Combinator, Google, and Plug & Play.

This assessment mirrors real problems our engineers solve daily. You will build an agent that performs precedent research across a corpus of Indian court judgments — a core capability of our platform.

The Problem

You are given a corpus of 50+ Indian court judgments **(link to the corpus of judgements:lexi_research_take_home_assessment_docs - provided as PDF files named DOC_001.pdf through DOC_050.pdf)** and a case brief describing a client's legal situation. Your task is to build an intelligent agent that can research this corpus and provide actionable legal analysis.

Case Brief

> **Client:** Mrs. Lakshmi Devi **Matter:** Motor accident claim — death of spouse
>
> Mrs. Lakshmi Devi's husband was killed in a road accident involving a commercial truck. The truck driver was operating the vehicle without a valid driving license at the time of the accident. The insurance company (National Insurance Co.) is denying the claim on the ground that the motor insurance policy is void due to the driver being unlicensed, and therefore they bear no liability to pay compensation.

Key Facts:

- The deceased was 42 years old at the time of the accident
- Monthly income: ₹35,000
- Dependents: wife (Mrs. Lakshmi Devi) and two minor children (ages 8 and 12)
- The truck was a commercial vehicle owned by a transport company
- The truck driver did not hold a valid driving license
- The insurance company is contesting liability, arguing the policy is void

What the Agent Must Do

The agent must be capable of handling general queries (as most agents should be able to), but its core capability should be deep precedent research. When given a research task, the agent should dynamically determine its own workflow rather than following hard-coded steps. For precedent research specifically, the output must include:

1. **Supporting Precedents** — Judgments from the corpus that support the client's case, with specific reasoning about which facts align and what legal principles each judgment establishes.

2. **Adverse Precedents** — Judgments that the opposing side could use against the client, with an honest assessment of the risk each poses and how they might be distinguished or countered.

3. **Strategy Recommendation** — Based on the precedent analysis, what arguments should be prioritized, what compensation range is realistic, and what risks the client should be aware of.

Critical Requirement: Flexibility

The agent must not be a hard-coded pipeline that only handles the specific case brief above. We will test your agent with different prompts and different research questions during evaluation. The agent should be able to handle general queries about the documents ("Which of these judgments involve commercial vehicles?") as well as deep research tasks ("Find precedents that support our argument on contributory negligence"). The architecture should support this range naturally, not through if-else branching.

Deliverables

1. Hosted Application

Deploy a web interface where we can interact with your agent. A Streamlit or Gradio app is perfectly fine — we are not evaluating frontend skills. The interface must allow us to input any prompt (not just the provided case brief) and see the agent's response. Intermediate reasoning steps must be visible — we want to see which documents the agent retrieved, how it ranked them, and how it arrived at its conclusions. Do not show only the final output.

2. GitHub Repository

A private GitHub repository shared with the handles. The repo should contain clean, well-structured code with a README that includes setup instructions. We will read your code.

3. Architecture Decision Record (ADR)

A written document (1–2 pages, included in the repo) explaining your design choices. This is one of the most important deliverables. Cover the following:

- Why you chose your specific architecture (agent framework, retrieval strategy, chunking approach)
- What tradeoffs you made and why
- How your agent decides when a simple query requires a straightforward answer vs. when it needs to run a deeper research workflow
- What you would change if the corpus were 5,000 documents instead of 50
- What you would change if you had another week

4. Evaluation Framework + Results

An automated evaluation framework that measures your agent's performance. See the Evaluation section below for details. Include the eval code in your repo and the results in your ADR or as a separate report.

Evaluation Framework

Build an evaluation framework that measures your agent's performance on the following four dimensions. We require at least one automated eval per dimension, but you are free to build more. The methodology is up to you — we are evaluating your judgment on how to measure agent quality, not prescribing a specific approach.

Dimension 1: Precision

Of the precedents your agent identifies as relevant, what percentage are actually relevant to the case? This measures whether your agent over-retrieves and presents irrelevant judgments as meaningful.

Dimension 2: Recall

Of the precedents that should have been found, what percentage did your agent actually find? Define your own methodology for measuring coverage — how do you know your agent isn't missing relevant judgments? We will evaluate your agent's recall against our own internal benchmark during review.

Dimension 3: Reasoning Quality

When your agent explains why a precedent applies or does not apply, does the reasoning hold up? This is harder to measure automatically, and we want to see how you approach evaluating something qualitative.

Dimension 4: Adverse Identification

Did your agent identify precedents that work against the client's case? A system that only finds favorable cases is dangerous in legal practice. This dimension specifically measures whether your agent can surface unfavorable precedents and honestly assess the risk they pose.

Note: After running your evals, include a brief written analysis of where your agent fails and what you would fix first. Be prepared to discuss this in your interview.

Technical Constraints

Required:

- **Language:** Python

Allowed:

- LangChain, LlamaIndex, or any other LLM/agent SDK
- Any vector database or retrieval framework
- Any hosting platform (Streamlit Cloud, Railway, Render, etc.)
- Open-source embedding models

Not Allowed:

- CrewAI or any similar drag and drop agent builders
- Hard-coded pipelines that only handle the provided case brief
- Submissions that require us to run local infrastructure to evaluate (the hosted URL must work)

On Using AI Tools

AI tools are encouraged. We expect you to use Claude, Cursor, Copilot, or any other AI tools you work with. We are an AI company — we would be concerned if you didn't use them.

We are evaluating your judgment, architecture decisions, and ability to reason about your system — not whether you can write boilerplate from memory. The ADR and the interview conversation are where we assess understanding. If you used AI tools to generate code, you should be able to explain every architectural choice in that code and reason about alternatives.

A Note on Legal Domain Knowledge

You are not expected to know Indian motor accident law. We are not evaluating legal accuracy. We are evaluating whether your system is well-architected, whether it extracts and uses

information from the documents faithfully, and whether you can reason about where it fails. A strong submission from an engineer with zero legal background will outperform a mediocre submission from someone who happens to know the law.

Timeline and Submission

We expect this to take 6–8 hours of focused work. You have 5 days from receiving this assessment to submit.

To submit:

1. Share the GitHub repository with the handles. The repo should contain your code, ADR, and evaluation framework as described in the Deliverables section.
2. Include the hosted URL in your README.
3. Ensure the hosted app is live and accessible at the time of submission.
4. Send an email to pradeep.kumaar@zyoin.com with the subject line:

[LEXI-BE-2026] Your Full Name - Research Assessment Submission

Include the repo link and hosted URL in the email body. Applications without this exact subject line format may be missed.

What Happens Next

We will review your submission and test your agent with the provided case brief as well as alternative prompts. If your submission moves forward, we will schedule a 30 to 45-minute technical interview where you will walk us through your architecture, discuss your eval results, and talk about where your agent fails and what you would improve.

Questions? Reach out to pradeep.kumaar@zyoin.com We're happy to clarify anything about the assessment.